

ISSUE DATE : 05-03-2026

SUBMISSION DATE:13-03-2026

Very Short Questions:

1. What is a cookie in PHP?

Ans: A cookie is a small piece of data stored on the client's browser by the server to remember user information across pages.

2. Which PHP function is used to create a cookie?

Ans: setcookie()

3. Where are cookies stored?

Ans: Cookies are stored in the user's browser (client-side).

4. What is the default lifetime of a session?

Ans: By default, a session lasts until the browser is closed or until the server session timeout (usually 1440 seconds / 24 minutes).

5. Which function is used to start a session?

Ans: session_start()

6. How do you access a cookie value in PHP?

Ans: Using the \$_COOKIE superglobal array.

7. Which superglobal variable is used for sessions?

Ans: \$_SESSION

8. What function is used to destroy a session?

Ans: session_destroy()

9. What does session_unset() do?

Ans: It removes all session variables but does not destroy the session itself.

10. What does flock() do in PHP?

Ans: It locks or unlocks a file to prevent simultaneous access by multiple users.

11. Which function is used to open a file?

Ans: fopen()

12. What does file mode "a" represent?

Ans: It opens a file for appending (writes data at the end of the file).

13. What is the purpose of mkdir()?

Ans: It is used to create a new directory.

14. Which function is used to delete a file?

Ans: unlink()

15. What does fgets() do?

Ans: It reads a single line from a file.

16. Which function checks if a file exists?

Ans: file_exists()

17. What does chmod() do?

Ans: It changes the file permissions.

18. What is the maximum upload size controlled by in PHP configuration?

Ans: It is controlled by upload_max_filesize and post_max_size in the php.ini file.

19. Which superglobal is used for file uploads?

Ans: \$_FILES

20. What is the difference between session and persistent cookies?

Ans: A session cookie is temporary and deleted when the browser closes, while a persistent cookie remains stored until its expiry time is reached.

Short Questions:

1. Explain cookies and their types.

Ans: A **cookie** is a small text file stored on the client's browser by a web server. Cookies are used to store user-specific information such as login credentials, user preferences, language settings, shopping cart data, and tracking information. Since HTTP is a stateless protocol, cookies help maintain user state between multiple page requests.

When a user visits a website, the server sends a cookie to the browser using the `setcookie()` function. The browser stores the cookie and sends it back to the server with every subsequent request to the same domain.

Types of Cookies:

1. Session Cookies

- Temporary cookies.
- Deleted automatically when the browser is closed.
- Used for short-term storage such as login sessions.

2. Persistent Cookies

- Stored in the browser for a specific time period.
- Remain even after closing the browser.
- Used for remembering login credentials or user preferences.

3. Secure Cookies

- Sent only over HTTPS connections.
- Protect sensitive information.

4. HTTP-Only Cookies

- Accessible only via HTTP protocol.
- Cannot be accessed using JavaScript.
- Helps prevent XSS attacks.

Advantages:

- Easy to implement
- Useful for personalization
- Reduces server load

Disadvantages:

- Limited storage size (~4KB)
- Security risks if not handled properly
- Can be disabled by users

2. Write syntax to create and delete a cookie in PHP.

Ans: Creating a Cookie

The `setcookie()` function is used to create a cookie. It must be called before any HTML output.

Syntax:

```
setcookie(name, value, expire, path, domain, secure, httponly);
```

Example:

```
setcookie("username", "John", time() + 3600, "/", "", false, true);
```

This creates a cookie:

- Name: username
- Value: John
- Expires in 1 hour
- Available across the entire website
- HTTP-only enabled

Deleting a Cookie

To delete a cookie, set its expiration time to a past time.

```
setcookie("username", "", time() - 3600, "/");
```

The browser removes the cookie automatically.

3. Explain how to read cookie values in PHP.

Ans: Cookies are accessed using the `$_COOKIE` superglobal array.

Example:

```
if(isset($_COOKIE["username"])) {  
    echo "Welcome " . $_COOKIE["username"];  
}
```

Important Points:

- Always check using `isset()` before accessing.
- Cookie values are strings.
- Avoid storing sensitive data directly in cookies.
- Sanitize output using `htmlspecialchars()` to prevent XSS.

4. What are sessions? How are they different from cookies?

Ans:

A **session** is a server-side method used to store user information across multiple pages. Unlike cookies, session data is stored on the server, and only a session ID is stored on the client.

Sessions are started using:

```
session_start();
```

Data is stored in:

```
$_SESSION["username"] = "John";
```

Difference Between Cookies and Sessions

Feature	Cookies	Sessions
Storage Location	Client Browser	Server
Security	Less Secure	More Secure
Data Size	Small (~4KB)	Large
Expiry	Set manually	Ends on timeout
Speed	Faster (client-side)	Slightly slower

Sessions are preferred for authentication and sensitive data.

5. Explain session lifecycle in PHP.

Ans:

The session lifecycle consists of several steps:

1. Session Start

```
session_start();
```

Creates a new session or resumes an existing one.

2. Session ID Creation

A unique session ID is generated and stored in a cookie (PHPSESSID).

3. Store Session Data

```
$_SESSION["user"] = "Admin";
```

4. Access Session Data

```
echo $_SESSION["user"];
```

5. Session Timeout

Session expires after a default timeout (usually 24 minutes).

6. Destroy Session

```
session_unset();
```

```
session_destroy();
```

This removes session data from the server.

6. Write steps to create a login system using sessions.

Ans:

1. Create a login form (username & password).
2. Validate user credentials (hardcoded or database).
3. Start session using session_start().
4. Store user data in session:
`$_SESSION["username"] = $username;`
5. Redirect to dashboard page.
6. On dashboard, check:
`if(!isset($_SESSION["username"])) {
 header("Location: login.php");
}`
7. Create logout page:
`session_destroy();
header("Location: login.php");`

This ensures secure user authentication.

7. Explain file permissions with examples (644, 755, 777).

Ans:

File permissions determine who can read, write, or execute files.

Format: Owner – Group – Others

Each permission:

- 4 = Read
- 2 = Write
- 1 = Execute

Examples:

644

- Owner: Read + Write
- Others: Read only

755

- Owner: Read, Write, Execute
- Others: Read and Execute

777

- Full access to everyone (Not recommended)

Use:

`chmod("file.txt", 0644);`

8. Describe file writing modes in PHP.

Ans:

When opening files with fopen(), modes define behavior:

Mode	Meaning
r	Read only
w	Write (overwrite)
a	Append
x	Create new file
r+	Read & Write
w+	Read & Write (overwrite)

Example:

```
$file = fopen("data.txt", "a");  
fwrite($file, "New Data\n");  
fclose($file);
```

9. Explain file locking and its importance.

Ans:

File locking prevents multiple users from accessing a file simultaneously.

Syntax:

```
flock($file, LOCK_EX);
```

Types:

- LOCK_SH → Shared lock (read)
- LOCK_EX → Exclusive lock (write)
- LOCK_UN → Unlock

Importance:

- Prevents data corruption
- Avoids race conditions
- Ensures data consistency

10. Write a PHP script to read file contents.

Ans:

```
<?php
```

```
$file = fopen("data.txt", "r");
```

```
if($file) {  
    while(!feof($file)) {  
        echo fgets($file) . "<br>";  
    }  
    fclose($file);  
} else {  
    echo "File not found.";  
}  
?>
```

11. Explain handling file uploads in PHP.

Ans:

Steps:

1. Create form:
`<form method="post" enctype="multipart/form-data">`
2. Access file using:
`$_FILES["file"]`
3. Validate file size:
`if($_FILES["file"]["size"] < 2000000)`
4. Move file:
`move_uploaded_file($_FILES["file"]["tmp_name"],`
`"uploads/".$filename);`

12. Describe how to restrict file types during upload.

Ans:

```
$allowed = ["jpg", "png", "pdf"];  
$ext = strtolower(pathinfo($_FILES["file"]["name"],  
PATHINFO_EXTENSION));
```

```
if(in_array($ext, $allowed)) {  
    echo "Allowed";  
}
```

Also validate MIME type for extra security.

13. Explain directory handling functions in PHP.

Ans:

Common functions:

- mkdir() → Create directory

- rmdir() → Remove directory
- scandir() → List files
- opendir() → Open directory
- readdir() → Read directory
- is_dir() → Check directory

Example:

```
$files = scandir("uploads/");
print_r($files);
```

14. What are security risks associated with file uploads?

Ans:

1. Malicious script uploads (.php)
2. File overwrite attacks
3. Large file DoS attacks
4. Path traversal attacks
5. Hidden executable files

Prevention:

- Restrict extensions
- Rename files
- Limit size
- Store outside public directory
- Disable execution permissions

15. Explain reading files incrementally with example.

Ans:

Reading incrementally improves memory efficiency.

Example:

```
$file = fopen("largefile.txt", "r");
```

```
while($line = fgets($file)) {
    echo $line . "<br>";
}
```

```
fclose($file);
```

Character-by-character:

```
while($char = fgetc($file)) {
    echo $char;
}
```

Useful for large files to avoid memory overload.

Long Questions:

1.Explain cookies in detail with syntax and example program

Ans:

Introduction

A **cookie** is a small text file stored on the client's browser by a web server. It is used to store user-related information such as login credentials, preferences, theme settings, shopping cart details, etc. Since HTTP is a stateless protocol, cookies help maintain state between multiple page requests.

Cookies are created using the `setcookie()` function in PHP and must be set before any HTML output.

Syntax of `setcookie()`

```
setcookie(name, value, expire, path, domain, secure, httponly);
```

Parameters:

- **name** → Cookie name
- **value** → Cookie value
- **expire** → Expiry time (Unix timestamp)
- **path** → Available directory
- **domain** → Domain name
- **secure** → True if HTTPS
- **httponly** → Accessible only via HTTP (not JavaScript)

Example Program

Step 1: Create Cookie

```
<?php
setcookie("username", "John", time() + 3600, "/", "", false, true);
?>
```

Step 2: Read Cookie

```
<?php
if(isset($_COOKIE["username"])) {
    echo "Welcome " . $_COOKIE["username"];
} else {
    echo "Cookie not set.";
}
?>
```

Advantages

- Simple to use
- Helps in personalization
- Reduces server processing

Disadvantages

- Limited storage (4KB)
- Less secure (client-side storage)
- Can be disabled

2. Describe how to create, read, modify, and delete cookies in PHP

Ans:

1 Create a Cookie

```
setcookie("user", "Admin", time()+3600, "/");
```

2 Read a Cookie

```
echo $_COOKIE["user"];
Always check:
if(isset($_COOKIE["user"])) {
    echo $_COOKIE["user"];
}
```

3 Modify a Cookie

To modify, simply reset with a new value:
setcookie("user", "John", time()+3600, "/");
The previous value is overwritten.

4 Delete a Cookie

```
setcookie("user", "", time()-3600, "/");
The cookie expires immediately.
```

3. Explain sessions in PHP with complete example of login/logout system

Ans:

Introduction

A session is a server-side storage mechanism used to store user data across multiple pages. Only the session ID is stored in the browser.

Session Functions

- session_start() → Start session
- \$_SESSION[] → Store data
- session_unset() → Remove variables
- session_destroy() → Destroy session

Example: Simple Login System

login.php

```
<?php
session_start();

if($_POST["username"] == "admin" && $_POST["password"] ==
"123") {
    $_SESSION["username"] = "admin";
    header("Location: dashboard.php");
}
?>
<form method="post">
Username: <input type="text" name="username">
Password: <input type="password" name="password">
<input type="submit" value="Login">
</form>
```

dashboard.php

```
<?php
session_start();

if(!isset($_SESSION["username"])) {
    header("Location: login.php");
}

echo "Welcome " . $_SESSION["username"];
?>
<a href="logout.php">Logout</a>
```

logout.php

```
<?php
session_start();
session_unset();
session_destroy();
header("Location: login.php");
?>
```

Advantages of Sessions

- Secure (server-side)
- Can store large data
- Ideal for authentication

4. Compare Cookies and Sessions in tabular form

Ans:

Feature	Cookies	Sessions
---------	---------	----------

Storage Location	Client	Server
Security	Less secure	More secure
Data Size	~4KB	Larger
Expiry	Defined manually	Timeout/browser close
Speed	Faster	Slightly slower
Usage	Preferences	Authentication

5.Explain file handling in PHP with examples of writing, reading, and appending data

Ans:

File Handling Functions

- fopen()
- fwrite()
- fread()
- fgets()
- fclose()

Writing (Overwrite Mode)

```
$file = fopen("data.txt", "w");
fwrite($file, "Hello World");
fclose($file);
```

Appending

```
$file = fopen("data.txt", "a");
fwrite($file, "New Line\n");
fclose($file);
```

Reading

```
$file = fopen("data.txt", "r");
while(!feof($file)) {
    echo fgets($file);
}
fclose($file);
```

6. Explain file locking mechanism and its importance in concurrent systems

Ans:

File locking prevents simultaneous access to files.

Syntax

```
flock($file, LOCK_EX);
```

Types

- LOCK_SH → Shared
- LOCK_EX → Exclusive
- LOCK_UN → Unlock

Importance

- Prevents data corruption
- Avoids race conditions
- Maintains consistency in multi-user systems

Example:

```
$file = fopen("data.txt", "a");
flock($file, LOCK_EX);
fwrite($file, "Secure Write\n");
flock($file, LOCK_UN);
fclose($file);
```

7.Describe the complete process of file uploading with validation

Ans:

Step 1: Create Form

```
<form method="post" enctype="multipart/form-data">
<input type="file" name="file">
<input type="submit">
</form>
```

Step 2: Handle Upload

```
$allowed = ["jpg","png","pdf"];
$filename = $_FILES["file"]["name"];
$ext = pathinfo($filename, PATHINFO_EXTENSION);

if(in_array($ext, $allowed) && $_FILES["file"]["size"] < 2000000) {
    move_uploaded_file($_FILES["file"]["tmp_name"],
"uploads/".$filename);
    echo "Uploaded Successfully";
} else {
    echo "Invalid File";
}
```

Validation Includes:

- File extension check
- File size check
- Rename file
- MIME validation

8. Explain directory management in PHP with examples

Ans:

Directory functions:

- mkdir() → Create directory
- rmdir() → Remove directory
- scandir() → List files
- is_dir() → Check directory

Example

```
mkdir("testfolder");
```

```
$files = scandir("testfolder");
print_r($files);
```

```
rmdir("testfolder");
```